Google Antigravity - QuizApp

FSL Version: V1.0.0

Introduction

Welcome to our coding challenge! This challenge is designed to test your ability to effectively leverage AI agents. You'll be expected to guide and collaborate with agents to design, build, and refine your solution. This is a great opportunity to showcase your technical expertise, strategic thinking, and creativity in working with intelligent tools. Good luck, and have fun exploring the power of agents!

Guidelines

You will create a screen recording video of yourself completing the challenge, then send us a link to the file via Google Drive. A few things to consider:

> **The recording must capture the entire coding challenge and not exceed 60 minutes. While there is some flexibility to go beyond this limit, any additional time over 60 minutes will result in penalties and may affect your application.**
>
> **You MUST NOT edit your video**, stop it and continue later, copy contents from hidden screens, or do anything that could be considered cheating.
>
> You should record your entire screen; you can use screen recording software like Loom, QuickTime, or something similar. Y**our computer clock should be visible throughout the video.**
>
> As you complete the challenge, please explain what you are doing. Walk us through your thinking, explain your decisions, etc.
>
> You cannot search for information from websites like Stackoverflow, blogs, forums, or similar; however, you **can** use the official documentation website of the technology.
>
> The recording must be done in one take without pauses or editing. **You MUST NOT stop or interrupt the recording at any point**; if you do, you will be disqualified.
>
> You should **start the recording after completing the project's environment and Agent setup**.

Here is a short clip from a recent coding challenge as an example of what your recording should look like (Example Video). It is from a React challenge, but it is the same for any challenge.

Please upload the video file to Google Drive and share an open link with us (we support .mp4 files smaller/with less than 4 GB).

AI Development Environment

This project will be built using Google Antigravity as the agentic IDE.

- Coding Agents
  - Google Antigravity
- Example of models you can use: (select based on task complexity)
  - Gemini 3 Pro
  - Claude Sonnet
  - GPT-OSS

You are free to choose your programming language, framework, and database.

AI-Assisted Development Expectations

This challenge evaluates your ability to **use AI as the primary developer**, not just an assistant. You should:

- **Prompt the AI to architect** the application structure and implementation plan
- **Have the AI write the code** for endpoints, models, business logic, and tests
- **Create agents or workflows** that autonomously implement features
- **Guide and iterate** through prompts rather than writing code manually

We're assessing: Your skill in directing AI tools to build functional software through effective prompting and task delegation.

Not expected: Manually writing code with AI only providing suggestions or reviews.

The Coding Challenge

The **AI Development Quiz App** is an educational product designed to help users test and reinforce their understanding of AI software development concepts, including agent design, prompt engineering, and workflow automation. You should build a RESTful API to manage quizzes, track user attempts, and calculate scores. This backend service focuses exclusively on server-side logic, data persistence, API design, and **asynchronous service communication**, leveraging your **AI coding assistant** to implement these functionalities.

General requirements

Build an API that helps users learn and test their knowledge of AI software development concepts with rich text content instead of plain text

Demonstrate RESTful API design and implementation

Implement proper data modeling and persistence

Support easy expansion with new quizzes or question sets

Handle business logic for quiz scoring and progress tracking

Implement asynchronous communication with external services

Write clean, maintainable code with appropriate error handling

Required API Endpoints

Quiz Management

- Retrieve all available quizzes (id, title, description only)
- Get full quiz details, including all questions and options (but NOT correct answers)
- Create a new quiz

Quiz Attempt & Submission

- Start a new quiz attempt for a user
- Submit answers and get results with scoring and feedback. Provide contextual performance feedback based on the user's score (e.g., 80%+ encouraging messages, 60-79% motivational feedback, <60% encouragement to improve)
- **Trigger asynchronous email notification** when a quiz is completed, sending the user a results summary

User Progress

- Get all quiz attempts for a specific user
- Get detailed results of a particular attempt (Attempt ID, User ID, Quiz ID and title, Timestamp, Overall score, and question-by-question breakdown with correctness and explanations)
- Get aggregate statistics for a user (total attempts, average score)

Persistence & Data Your API should handle and store:

- Quiz information (title, description, questions)
- Questions with multiple choice options, correct answers, and explanations
- User identifiers and contact information (email)
- Attempt records (which user took which quiz, when, and their score)
- Individual answer submissions for each attempt

Notification tracking (status, timestamps)

Asynchronous Communication Requirement

When a user completes a quiz, the system must **asynchronously** communicate with an email notification service to send a results summary.

Your implementation should:

- Demonstrate asynchronous communication patterns appropriate for your architectural approach
- Mock the email service **(you don't actually need to send emails)**
- Handle the scenario where the quiz submission should not fail if the email service is unavailable
- Track notification status appropriately

Email notification should include:

- User's name and email
- Quiz title
- Score (correct answers / total questions)
- Percentage score
- Performance feedback message
- Timestamp of completion

Consider and decide:

- Which asynchronous communication approach best fits your architecture (message queues, background jobs, webhooks, event-driven patterns, etc.)
- How to ensure reliability and fault tolerance
- How to handle retries and failure scenarios
- How to track and monitor notification status

Production-Ready Considerations

Beyond the core requirements, you should apply production-ready practices based on your professional judgment. Consider which of the following areas are most critical for this application and implement accordingly:

Architecture & Design Patterns

- Separation of concerns (layers, modules, services)
- Design patterns that improve maintainability
- Code organization strategies
- Dependency injection or inversion of control

Data Modeling & Persistence

- Database schema optimization (indexes, relationships, normalization)
- Query performance considerations
- Data validation strategies
- Migration and seeding approaches

Type Safety & Code Quality

- Strong typing vs dynamic typing trade-offs
- Interface definitions and contracts
- Error handling patterns
- Code documentation approaches

Performance & Scalability

- Caching strategies
- Connection pooling
- Stateless vs stateful design
- Horizontal scaling considerations

Security & Reliability

- Input sanitization and validation
- Authentication and authorization approaches
- Rate limiting or throttling
- Graceful error handling and circuit breakers

Testing & Quality Assurance

- Test coverage strategy
- Mocking and stubbing approaches
- Integration vs unit testing decisions
- Test data management

We're evaluating your ability to make informed trade-offs between:

- Simplicity vs feature completeness
- Development speed vs code quality
- Immediate functionality vs future maintainability
- Practical solutions vs over-engineering

Antigravity-Specific Requirements

Your submission must demonstrate use of these Antigravity features:

Agent Manager: Orchestrate parallel agent tasks (e.g., one builds endpoints while another writes tests)

Artifacts: Include the exported Task List and Implementation Plan artifacts in an `/artifacts` folder

Skills: Create project-specific skills to define code patterns, conventions, or workflows

MCP Integration: Connect MCP servers to extend agent capabilities (e.g., database, GitHub, Jira)

Development Mode: Document your chosen mode (Agent-Driven, Agent-Assisted, or Review-Driven) and justify in README

Example Scenario

API Usage Example:

A user sends a request to retrieve all available quiz categories from the API

The API returns three quiz categories. The user selects "Agent Fundamentals"

The user sends a request to start a new quiz attempt for the "Agent Fundamentals" quiz

The API responds with the attempt details and all 5 questions with their options (but not the correct answers)

The user reviews the questions and prepares answers for all 5 questions

The user submits the answers to the API

The API processes the answers, calculates a score of 4/5 (80%), and returns:

- Overall score and percentage
- Performance feedback: "Good job! You're getting there!"
- For each question: correct/incorrect status and explanation
- The API asynchronously triggers a notification to send the user an email with the results

The email notification service (mocked) processes the request and simulates sending the email

The user sends a request to view quiz attempt history

The user decides to retake the quiz and sends another request to start a new attempt for the same quiz

The API creates a new attempt record while preserving the previous attempt history

Technical Requirements

Backend API

Database integration (your choice)

Asynchronous communication mechanism (your choice of approach)

Data persistence across requests

- Input validation and error handling
- Mock email service implementation
- Interactive API Documentation (Swagger/OpenAPI or equivalent) with endpoint descriptions, request/response schemas, and example payloads

Technology Stack Selection: You are expected to select your technology stack based on your architectural and technical decisions. Be prepared to justify your choices regarding:

- Programming language and framework
- Database type and configuration
- Asynchronous communication approach
- Testing frameworks and tools
- Any additional libraries or services

Acceptance Criteria

- All required API endpoints are implemented and functional
- Interactive API documentation is accessible, and all endpoints are testable
- Quiz submission triggers asynchronous email notifications
- Mock email service demonstrates async communication patterns
- User attempt history is persisted and retrievable via API
- Proper error handling and validation across all endpoints
- Code demonstrates production-ready practices and sound architectural decisions
- **Seed data**: At least 2 pre-loaded quiz categories with 5+ questions each
- **Basic unit tests** cover core business logic (scoring, feedback calculation, async messaging)